

Project 12 (Bonus) — MCPResearch-Multi-Agent-Pipeline-Exposed-As-One-MCP-Tool

Title: MCPResearch Multi Agent Pipeline Exposed As One MCP Tool

Type: MCP Server (1 Tool, wrapping a 2-agent pipeline) · **Stack:** Python, official `mcp` SDK (MCPServer), LangChain (`langchain-openai`), `asyncio` · **Level:** Advanced (bonus project)

Tagline: A Researcher agent and a Fact-Checker agent loop until a summary is approved — the whole loop hides behind one MCP tool, `deep_research(topic)` , so any MCP client calls it like a single plain function.

Overview

MCPResearch wraps an entire two-agent pipeline — a Researcher that drafts a summary and a Fact-Checker that reviews it for real errors — behind a single MCP tool, `deep_research(topic)` . A calling client sees only a plain function call and gets back a fact-checked summary; it has no idea a bounded revision loop with multiple LLM calls ran underneath. This is the "agent-as-a-service" pattern used in production systems: a whole internal workflow exposed as one capability other tools or teams can call, without needing to understand or depend on its internal steps. The project also solves the two real problems that pattern creates on its own: giving a caller feedback on a slow, multi-round call, and falling back to a clearly-marked partial answer instead of hanging or crashing when the pipeline runs too long or fails.

Features

- Exposes a full Researcher/Fact-Checker revision loop as a single MCP tool, `deep_research(topic)`
- Bounds the revision loop with a hard round cap so it can't loop forever on a topic that never fully converges
- Returns a per-round revision trace alongside the final summary, so a caller can see what happened without a live progress protocol
- Wraps the whole pipeline call in a timeout, falling back to a clearly-marked partial result instead of hanging
- Catches an internal pipeline failure and turns it into a clear, marked-unconfirmed result instead of crashing or returning nothing
- Proves the pipeline's own logic (Researcher, Fact-Checker) correct as a standalone Python program before any MCP layer wraps it

Tech Stack

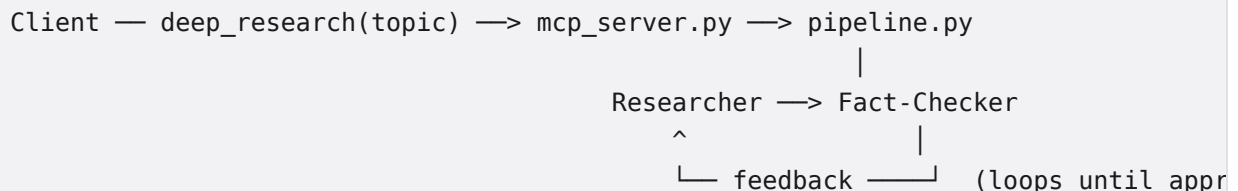
- Python
- Official `mcp` SDK (MCPServer)
- LangChain (`langchain-openai`)
- `asyncio`

Prerequisites

- Understand how to build an MCP server with MCPServer and expose tools
- Familiar with the generator-critic pattern: one agent drafts, another reviews and sends back feedback for revision
- Comfortable with Python `asyncio` , including timeouts and running a blocking call in `asyncio.to_thread()`
- Know how to call a chat model through LangChain (`langchain-openai`)

Architecture

`pipeline.py` runs a bounded loop between two agents: a Researcher (`researcher.py`) that drafts and revises a summary, and a Fact-Checker (`fact_checker.py`) that reviews each draft for real errors and either approves it or sends back specific issues to fix. This loop is proven correct as a plain Python program first, with no MCP involved. `mcp_server.py` then wraps a single call to `run_pipeline()` as one MCPServer tool, `deep_research(topic) -> str` — the MCP layer never sees the two agents or the loop between them, only a finished result. To give a caller visibility into a slow, multi-round call without a more complex client-side protocol, the tool's return value is a structured plain-text result: a per-round trace plus the final summary. The whole pipeline call runs inside `asyncio.to_thread()` wrapped in `asyncio.wait_for()` , so a timeout or an unexpected failure returns a clearly-marked partial result instead of hanging the caller or crashing.



Setup (do this once, before Step 1)

These are the very first commands to run. You don't need to read anything else first.

```
cd project_12_mcpresearch
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install "mcp[cli]" langchain langchain-openai python-dotenv pydantic
pip freeze > requirements.txt
```

Create a file called `.env` in this folder (never commit this file to git):

```
LOG_LEVEL=INFO
OPENAI_API_KEY=sk-your-real-key-here
MAX_REVISION_ROUNDS=3
DEEP_RESEARCH_TIMEOUT_SECONDS=60
```

Create a second file, `.env.example`, with the same keys but a placeholder for the real secret:

```
LOG_LEVEL=INFO
OPENAI_API_KEY=sk-your-key-here
MAX_REVISION_ROUNDS=3
DEEP_RESEARCH_TIMEOUT_SECONDS=60
```

`OPENAI_API_KEY` is a real secret — only `.env` should have your real value, and only `.env.example` goes in git.

Add `.venv/` and `.env` to your `.gitignore` before your first commit.

`mcp[cli]` installs the **MCP Inspector**, the same tool used to test any small MCP server. Once `mcp_server.py` exists (from Step 2 on), you can test `deep_research` by hand with `mcp dev mcp_server.py`, without writing a client script.

Troubleshooting

Problem	Fix
The Fact-Checker approves everything on the first try, so the loop never actually runs more than once	Your Fact-Checker's prompt is too soft. Write real scoring criteria into it (dates, names, invented facts, overconfident claims) — a vague critic approves almost everything
The pipeline loops the full <code>MAX_REVISION_ROUNDS</code> every single time, never converging	The Researcher isn't actually using the feedback. Check that <code>feedback</code> is really being passed into the prompt each round, and that the prompt tells it to fix only the named issues, not rewrite from scratch
<code>deep_research</code> hangs the MCP Inspector for a long time with no	Return a revision trace alongside the answer instead of only the bare final string

Problem	Fix
feedback	
A slow topic makes <code>deep_research</code> hang seemingly forever	Add a real timeout and a "best draft so far" fallback, so the tool always returns <i>something</i>
The client hangs and never gets a reply, even though your code looks right	Something is writing to stdout directly (a stray <code>print()</code>) — stdout is the protocol channel for a stdio-based MCP server. Route all logging through <code>logging_setup.py</code> 's logger, never <code>print()</code>
The MCP tool still "hangs" even after adding <code>asyncio.wait_for()</code>	You wrapped a blocking, synchronous call directly with <code>wait_for</code> — a plain sync function inside an <code>async def</code> still blocks the event loop. Run it with <code>asyncio.to_thread()</code> first, then wrap that in <code>wait_for()</code>

Usage Example

Use this topic, or make your own, but keep the shape the same: something with a real, checkable fact buried in it, so the Fact-Checker's rejection isn't hypothetical.

Topic: `"Retrieval-Augmented Generation (RAG)"`

Round 1: The Researcher drafts something like: *"Retrieval-Augmented Generation (RAG) was introduced by OpenAI in 2019 to let language models pull in outside documents before answering."* The Fact-Checker rejects it: `REJECTED` — issue: *"RAG was introduced by Lewis et al. at Facebook AI Research (now Meta AI) in 2020, not OpenAI in 2019."*

Round 2: The Researcher redrafts using that exact feedback: *"Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. at Meta AI (then Facebook AI Research) in 2020. It lets a language model retrieve relevant documents before generating an answer, instead of relying only on what it memorized during training."* The Fact-Checker approves it.

What `deep_research("Retrieval-Augmented Generation (RAG)")` returns (from Step 3 onward) — the final summary, plus a short trace of what happened:

```
Deep research on: Retrieval-Augmented Generation (RAG)

Revision trace:
  Round 1: rejected
    - RAG was introduced by Lewis et al. at Facebook AI Research (now Meta AI) in
  Round 2: approved

Status: approved after 2 round(s).
```

Summary:

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. at Meta AI (then Facebook AI Research) in 2020. It lets a language model retrieve relevant documents before generating an answer, instead of relying only on what it memorized during training.

Two more topics worth testing on purpose: - A topic with almost nothing checkable in it (e.g. "my opinion of the color blue") — watch what happens when the Fact-Checker has nothing real to verify. Does it approve immediately, or nitpick forever? This is the case that proves your revision cap actually matters. - A topic you deliberately make the pipeline slow on, or a fake network failure inside `run_researcher` — this is what Step 4's timeout and fallback path are for. If you never test the unhappy path, you don't actually know it works.

Jump to: [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#)

How To Build This — Step by Step

Step 1 — The Two-Agent Pipeline, Running Standalone (No MCP Yet)

Project: MCPResearch-Multi-Agent-Pipeline-Exposed-As-One-MCP-Tool — Step 1 of 4: The Two-Agent Pipeline, Running Standalone (No MCP Yet)

What this step does: builds the Researcher, the Fact-Checker, and the loop between them as a plain Python program you run directly — no MCP anywhere in this step. It proves the pipeline itself works before anything wraps it. **Why this step matters:** if the pipeline's own logic is broken, wrapping it in MCP later would only hide that bug behind a protocol, not fix it. Proving it alone first means any bug you hit in Step 2 onward can never be blamed on the Researcher or the Fact-Checker. **When you'll hit this for real:** any time you're about to expose something over a protocol (MCP, a REST API, a queue) — prove the thing underneath actually works on its own first, the same instinct any MCP server project uses for its own data store before adding Tools on top of it. **Helpful background (if needed):** the Generator → Critic → Revision multi-agent pattern and its "without a hard limit" warning; why multi-agent systems cost more time and money than a single call.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Build `config.py` and `logging_setup.py` (or reuse them from an earlier project if you already have working copies). `LOG_LEVEL` , `OPENAI_API_KEY` , and `MAX_REVISION_ROUNDS` are all you need this step. 2. Write `researcher.py` : a `Draft` type (topic + summary), and `run_researcher(topic: str, feedback: str | None = None) -> Draft` . When `feedback` is given, the prompt must tell the model to fix exactly the named issues, not rewrite the whole thing from scratch — this is what makes Round 2 actually different from Round 1, the same lesson any Generator/Writer agent teaches with its own `feedback` parameter. 3. Write `fact_checker.py` : a `FactCheckVerdict` type (`approved: bool` , `issues: list[str]`), and

`run_fact_checker(draft: Draft) -> FactCheckVerdict` . Write real scoring criteria into its prompt on purpose — wrong dates, invented facts, overconfident claims — a vague prompt approves almost everything, and then you'll never actually see the loop run more than once. 4. Write `pipeline.py` : `run_pipeline(topic: str, max_rounds: int) -> PipelineResult` , where `PipelineResult` holds the final summary, whether it was approved, and a list of what happened each round. Loop: Researcher drafts → Fact-Checker checks → if approved, stop; if not, feed the issues back to the Researcher as `feedback` and go again — up to `max_rounds` times, then stop regardless, exactly like the standard warning about a multi-agent loop that never converges. 5. Write `main.py` that calls `run_pipeline()` directly on 2-3 topics (including one deliberately hard to fact-check) and prints every round's result. Confirm the loop actually improves a deliberately wrong first draft, and confirm it stops cleanly at the cap on the hard topic instead of looping forever.

Your files after Step 1:

```
project_12_mcpresearch_agentic_mcp_tool/
├── .env / .env.example      → LOG_LEVEL, OPENAI_API_KEY, MAX_REVISION_ROUNDS, DEEP
├── config.py                → load_config() -> Config
├── logging_setup.py         → get_logger(name)
├── researcher.py            → Draft, run_researcher(topic, feedback=None) ->
├── fact_checker.py          → FactCheckVerdict, run_fact_checker(draft) ->
├── pipeline.py              → RevisionRound, PipelineResult, run_pipeline
└── main.py                  → runs the pipeline standalone on test top
```

Step 2 — Wrap It as a Single MCP Tool

Project: MCPResearch-Multi-Agent-Pipeline-Exposed-As-One-MCP-Tool — Step 2 of 4: Wrap It as a Single MCP Tool

What this step does: builds `mcp_server.py` , exposing `deep_research(topic: str) -> str` as one `MCPServer` tool. The tool's whole body is a call to Step 1's `run_pipeline()` — nothing about the pipeline changes. **Why this step matters:** this is the actual lesson of the project — the MCP layer is thin. It does not know, and does not need to know, that multiple agents ran underneath. Writing this step is what proves that claim in real code instead of just an idea. **When you'll hit this for real:** any time you take something that already works (a script, a pipeline, a whole system) and expose it as one callable unit for other tools to use, without those tools needing to understand its internals. **Helpful background:** what MCP actually is and the problem it solves; how MCP servers and clients actually connect; why MCP matters for multi-agent systems — that last idea describes exactly what you're doing in this step.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write `mcp_server.py` : a `MCPServer("deep-research-service")` instance, and one tool, `deep_research(topic: str) -> str` , whose body is exactly two lines — call `run_pipeline(topic, max_rounds=config.max_revision_rounds)` , then return

`result.final_summary` . Resist the urge to add anything cleverer yet — that's Step 3's job. 2. Write a clear tool description (a real docstring) — the idea that "a tool's description is really a prompt" applies here just as much as it did to a small tool, maybe more, since this tool's name doesn't make its cost (multiple LLM calls, possibly slow) obvious to whoever's calling it. 3. Test it with the MCP Inspector: `mcp dev mcp_server.py` , then call `deep_research` with your Real Example topic. Confirm you get back the plain final summary — no trace yet, that's Step 3. 4. Confirm `mcp.run(transport="stdio")` is the last line that actually executes, and that nothing in `researcher.py` , `fact_checker.py` , or `pipeline.py` ever calls `print()` — same stdout rule as any stdio-based MCP server, now with more files that could accidentally break it.

Your files after Step 2:

```
project_12_mcpresearch_agentic_mcp_tool/
├── .env / .env.example
├── config.py
├── logging_setup.py
├── researcher.py                (unchanged from Step 1)
├── fact_checker.py             (unchanged from Step 1)
├── pipeline.py                 (unchanged from Step 1)
├── main.py                     (unchanged from Step 1)
└── mcp_server.py               → MCPServer("deep-research-service");
```

Step 3 — Report Internal Progress Without Breaking the One-Tool Illusion

Project: MCPResearch-Multi-Agent-Pipeline-Exposed-As-One-MCP-Tool — Step 3 of 4: Report Internal Progress Without Breaking the One-Tool Illusion

What this step does: changes `deep_research` 's return value so it includes a short trace of what happened each round, alongside the final summary — not just the bare final string Step 2 returned.

Why this step matters: a caller waiting on `deep_research()` through 2-3 revision rounds currently gets no feedback at all until it's fully done. For a slow tool, that's a bad experience — the caller can't tell "still working" from "stuck." This step decides, on purpose, how to fix that without turning one simple tool call into a more complicated multi-step protocol. **When you'll hit this for real:** any tool wrapping a slow, multi-step process — a long-running job, a multi-agent pipeline, a batch operation — needs a real answer to "what does the caller see while this runs, and what does it see once it's done?" **Helpful background:** how MCP servers and clients actually connect — the transport `deep_research` 's result would have to travel over; shared state design — what goes in shared state and what doesn't, the same "what's worth surfacing vs. what's just noise" decision, one level up.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

The decision to make, and the reasoning behind it: the official `mcp` SDK does have a live progress-reporting mechanism, but it only works end to end if the specific client calling your tool

asked for progress updates and is actively listening for them — most simple clients (including a basic one you might write yourself) don't wire that up, and even when a client does, nothing about a progress *notification* sticks around afterward as part of the actual result. This project uses a **structured result** instead: `deep_research` still returns one plain string (the interface doesn't change at all), but that string now contains a short revision-history trace above the final answer. It works with every MCP client, with no special listening code required, and — unlike a stream of progress notifications — it's still there if you look at the result five minutes later.

What to do: 1. In `mcp_server.py`, write `format_result(result: PipelineResult) -> str`: a plain text layout with the topic, one line per round (`Round 1: rejected` plus its issues, `Round 2: approved`), a status line (approved after N rounds, or "NOT approved — hit the N-round cap"), and the final summary at the bottom. Match the shape in this README's "A Real Example" section. 2. Update `deep_research` to call `format_result(result)` instead of returning `result.final_summary` directly. 3. Test with the Inspector: call `deep_research` on your Real Example topic and confirm the trace shows both rounds, including the rejection issue from Round 1. 4. Test on the "almost nothing checkable" topic from the Real Example section too — confirm the trace clearly shows whether it converged quickly or hit the cap, so a caller reading the result can tell the difference between "this was fine" and "this struggled."

Your files after Step 3:

```
project_12_mcpresearch_agentic_mcp_tool/
├── .env / .env.example
├── config.py
├── logging_setup.py
├── researcher.py
├── fact_checker.py
├── pipeline.py                (unchanged from Step 1)
├── main.py                   (unchanged from Step 1)
└── mcp_server.py             → adds format_result(); deep_research(
```

Step 4 — A Timeout and a Partial-Result Fallback

Project: MCPResearch-Multi-Agent-Pipeline-Exposed-As-One-MCP-Tool — Step 4 of 4: A Timeout and a Partial-Result Fallback

What this step does: adds a hard timeout around the whole pipeline call, and makes `deep_research` return something useful — the best draft found so far, clearly marked as unconfirmed — instead of hanging the caller or returning nothing, if the pipeline runs too long or a step fails outright. **Why this step matters:** `MAX_REVISION_ROUNDS` (Step 1) stops the *loop* from running forever. It does not stop one *round* from taking a long time — a slow model response, a stuck network call — and it does nothing at all if `run_researcher` or `run_fact_checker` raises an exception. A tool with no timeout and no failure handling can hang a real client indefinitely, or crash with no useful information at all. **When you'll hit this for real:** any tool call — MCP or

otherwise — that wraps something slow or unreliable needs both of these: a ceiling on how long you'll wait, and a real decision about what "the best answer I can give right now" looks like when that ceiling is hit. **Helpful background:** how errors propagate between agents; the Generator → Critic → Revision pattern's "without a hard limit" trade-off, now applied to wall-clock time instead of just round count.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Add `DEEP_RESEARCH_TIMEOUT_SECONDS` to `config.py`. Write `exceptions.py` → `PipelineStepFailedError(Exception)`. In `pipeline.py`, wrap the Researcher/Fact-Checker calls in a `try / except`, and re-raise anything unexpected as `PipelineStepFailedError` with a clear message naming which round failed — the same "turn a raw exception into a clean, informative error" idea that applies to any tool call, now applied to one pipeline round. 2. Give `run_pipeline()` an optional `progress_sink` argument — a callback invoked with each `RevisionRound` right after it's produced. This is what lets `mcp_server.py` see partial progress even if the whole call never finishes. 3. Make `deep_research` an `async def`. Inside it, run the pipeline with `asyncio.to_thread(run_pipeline, topic, config.max_revision_rounds, record_round)`, wrapped in `asyncio.wait_for(..., timeout=config.timeout_seconds)`. Catch `asyncio.TimeoutError` and `PipelineStepFailedError` separately, and in both cases return a clearly-marked partial result built from whatever `record_round` collected before the failure — never a bare error, never a hang. 4. **Know the real limitation before you test it:** `asyncio.wait_for()` stops *waiting*, it does not kill the background thread. A truly stuck `run_pipeline()` call keeps running after your timeout fires — it's just orphaned. This is a genuine, honest limit of this approach, not a bug to chase down; a real production system would run the pipeline in a separate process it can actually terminate, or design the loop to check a cancellation flag between rounds. Say this out loud once you've built it — it's a real trade-off, not something to paper over. 5. Test all three unhappy paths: a topic you make artificially slow (confirm the timeout fires and you get a marked-unconfirmed partial answer, not a hang), a forced exception inside `run_researcher` (confirm you get a marked-unconfirmed partial answer, not a crash or an empty string), and the happy path once more (confirm nothing about the normal result changed).

Your files after Step 4 (final):

```
project_12_mcpresearch_agentic_mcp_tool/
├── .env / .env.example           → adds DEEP_RESEARCH_TIMEOUT_SECONDS
├── config.py                    → now also loads DEEP_RESEARCH_TIMEOUT_SECONDS
├── logging_setup.py
├── exceptions.py                → PipelineStepFailedError(Exception)
├── researcher.py                (unchanged from Step 1)
├── fact_checker.py              (unchanged from Step 1)
├── pipeline.py                  → run_pipeline() now takes an
├── main.py                     (unchanged from Step 1)
└── mcp_server.py               → deep_research() is now a
```

Final Deliverable: MCPResearch-Multi-Agent-Pipeline-Exposed-As-One-MCP-Tool — a real MCP server exposing one Tool, `deep_research`, whose body runs a bounded Researcher/Fact-Checker revision loop, reports a per-round trace alongside the final answer, and falls back to a clearly-marked partial result instead of hanging or crashing when the pipeline runs too long or a step fails.

Checklist Before You Call This Done

- ☐ `researcher.py` and `fact_checker.py` work correctly on their own, tested through `main.py`, before any MCP code exists
- ☐ The revision loop actually improves a deliberately wrong first draft, and stops cleanly at `MAX_REVISION_ROUNDS` on a topic that never fully converges
- ☐ `deep_research` is a single MCP tool — the Inspector shows exactly one tool, not one per agent
- ☐ `deep_research`'s result includes a per-round revision trace, not just the bare final summary
- ☐ A slow or stuck pipeline run times out, and still returns a clearly-marked, useful partial answer instead of hanging the client
- ☐ A forced failure inside a pipeline step is caught and turned into a clear, marked-unconfirmed result, never a crash or an empty string
- ☐ No `print()` anywhere in `researcher.py`, `fact_checker.py`, `pipeline.py`, or `mcp_server.py` — only the logger
- ☐ You can explain, out loud, why this is one MCP tool wrapping two agents instead of two MCP tools a client drives itself

Status

Not started. Track your own progress however works for you.